

VirtuEx 시스템 감사 보고서 (23차)

VirtuEx System Audit Report (23rd)

- 감사일: 2026-03-24
- 감사 범위: API 보안, 입력 검증, 에러 처리, 인증/인가, DB 무결성, 서비스 간 통신, 로깅/모니터링, 프론트엔드 보안, HTTP 보안 헤더, WebSocket 보안, IDOR, 감사 로깅, httpOnly 쿠키 마이그레이션 회귀 검증
- 감사 방법: 전체 소스 코드 정적 분석 + API 흐름 추적 + 보안 취약점 패턴 매칭 + 23차 강화 기준 적용
- 심각도 기준: Critical(즉시 수정) / High(1주 내) / Medium(2주 내) / Low(개선 권장)
- 비고: 본 보고서는 22차 대비 수정 현황을 검증하고, httpOnly 쿠키 마이그레이션 회귀 및 WebSocket 인증 토큰 처리를 중점 점검합니다

이전 감사 대비 수정 현황 (22차 → 23차)

이전 이슈	상태	비고
[SEC-H-02] API Gateway 로그아웃 시 access_token 쿠키 미삭제	수정됨	auth-proxy.controller.ts:189-190에서 <code>res.clearCookie('refresh_token', { path: '/api/auth' })</code> + <code>res.clearCookie('access_token', { path: '/' })</code> 두 쿠키 모두 삭제 확인
[VAL-M-02] 주문 수량 프론트엔드 소수점 자릿수 미제한	미수정	OrderForm.tsx:304에서 직접 입력 시 여전히 소수점 제한 없음
[ERR-M-01] 주문 프록시 거래 알림 실패 시 내부 에러 메시지 로깅	미수정	order-proxy.controller.ts:62에서 <code>e.message</code> 로그 노출 유지
[ERR-M-02] any 타입 사용에 의한 타입 안전성 결여	미수정	portfolio-proxy.controller.ts 전반에서 (<code>req as Record<string, any></code>). <code>user?.id</code> 패턴 14개소, <code>ai-proxy.controller.ts:48</code> , <code>order-proxy.controller.ts:164-176</code> 유지
[ERR-L-01] 프론트엔드 catch 블록 에러 무시 패턴	미수정	portfolio/page.tsx:99,113에서 <code>catch { // Error handled by query client }</code> 패턴 유지
[CSP-M-01] CSP unsafe-inline + strict-dynamic 공존	미수정	next.config.ts:9에서 프로덕션 CSP 동일 설정 유지
[AUTH-M-01] 호가창 엔드포인트 인증 불필요 노출	미수정	GET /api/orders/book/:symbol에 인증 가드 없음
[AUTH-L-01] 공개 포트폴리오 API 보유 자산 상세 노출	미수정	절대 수량 마스킹 미적용

[SVC-M-01] 서비스 간 HTTP 통신 HTTPS 미적용	미수정	proxy.service.ts:71-77에서 http:// 유지
[AUD-M-01] 비밀번호 재설정 감사 로깅 불충분	미수정	auth.service.ts:704에서 성공 시 logger.log만 기록, 실패 시 별도 로그 없음
[FE-M-01] RichEditor 이미지 업로드 서버 실패 시 Base64 폴백	미수정	RichEditor.tsx:144-147에서 Base64 data URL 폴백 유지
[FE-M-02] 주문 폼 displayCurrentPrice 기반 최대 수량 계산 편차	미수정	OrderForm.tsx:321에서 portfolio.cashBalance / displayCurrentPrice 계산 유지
[MISC-L-01] 공지사항 Gateway AdminRolesGuard 미적용	미수정	AnnouncementProxyController에 AdminRolesGuard 여전히 없음
[VAL-L-01] auth-proxy 컨트롤러 body 타입 unknown 전달	미수정	auth-proxy.controller.ts에서 @Body() body: unknown 패턴 유지

요약 (Executive Summary)

카테고리	Critical	High	Medium	Low	합계
A. 인증/토큰 보안	0	1	1	0	2
B. 입력 검증	0	0	1	1	2
C. 에러 처리/정보 노출	0	0	2	1	3
D. XSS/CSP 방어	0	0	1	0	1
E. 인가/권한 검증	0	0	2	2	4
F. 서비스 간 통신	0	0	1	0	1
G. 감사 로깅	0	0	1	0	1
H. 기타	0	0	2	0	2
I. WebSocket 보안	0	0	1	0	1
J. Git/이력 보안 (보충 감사)	0	0	1	1	2
K. 파일 업로드/프록시 보안 (보충 감사)	0	0	0	1	1
합계	0	1	13	5	19

A. 인증/토큰 보안 — 2건

[SEC-H-03] 응답 바디에 accessToken 노출 — httpOnly 쿠키 보안 우회 (High, 23차 신규)

- **현상:** auth.controller.ts:158-159 의 verifyLoginSms 및 auth.controller.ts:194-195 의 refresh 엔드포인트에서 액세스 토큰을 httpOnly 쿠키로 설정하는 동시에 응답 바디(data.accessToken)에도 포함하여 반환. 주석에 @deprecated: cookie auth preferred 로 표기되어 있으나, WebSocket 호환을 위해 유지 중. 프론트엔드 useAuthTokenRecovery.ts:55 와 api.ts:78 에서 이 바디 토큰을 읽어 Zustand 메모리에 저장. XSS 발생 시 응답 바디의 토큰을 가로채거나, Zustand 스토어의 accessToken 값을 읽어 탈취 가능하여 httpOnly 쿠키의 보안 이점이 부분적으로 무효화됨
- **위치:** backend/services/user-auth/src/presentation/controllers/auth.controller.ts:158,194 , frontend/src/hooks/useAuthTokenRecovery.ts:55 , frontend/src/lib/api.ts:78
- **영향:** XSS 공격자가 useAuthStore.getState().accessToken 으로 메모리 내 토큰을 직접 읽거나, refresh 응답을 인터셉트하여 토큰 탈취 가능. httpOnly 쿠키만 사용 시 불가능한 공격 벡터가 열려 있음
- **권장:** WebSocket 인증을 쿠키 기반으로 전환(Socket.IO extraHeaders + withCredentials)하여 응답 바디에서 토큰 제거. 전환 전까지는 응답 바디 토큰의 만료 시간을 짧게(예: 2분) 설정하는 것을 검토
- **심각도:** High

[SEC-M-01] /api/auth/refresh 엔드포인트 Rate Limit 미적용 (Medium, 23차 신규)

- **현상:** auth-proxy.controller.ts:153 의 refresh 엔드포인트에 @Throttle 데코레이터가 없음. 같은 컨트롤러의 다른 모든 POST 엔드포인트(register, login, sms/send 등)에는 Rate Limit이 적용되어 있으나, refresh 는 제한 없음. useAuthTokenRecovery.ts 가 페이지 새로고침마다, api.ts 인터셉터가 401 응답마다 호출하므로 정상 사용에서도 빈번하지만, 악의적 반복 호출에 대한 방어가 없음
- **위치:** backend/services/api-gateway/src/proxy/auth-proxy.controller.ts:153
- **영향:** 탈취된 리프레시 토큰으로 무한 토큰 갱신이 가능하며, 리프레시 토큰 회전(rotation) 전에 대량 액세스 토큰 발급으로 서비스 부하 유발
- **권장:** @Throttle({ default: { ttl: 60000, limit: 30 } }) 적용 (정상적인 자동 갱신을 허용하면서도 악의적 남용 차단)
- **심각도:** Medium

B. 입력 검증 — 2건

[VAL-M-02] 주문 수량 프론트엔드 소수점 자릿수 미제한 (Medium, 미수정)

- **현상:** OrderForm.tsx 에서 수량 직접 입력 시 소수점 20자리 이상 입력 가능. 비율 버튼 사용 시 .toFixed(8) 로 포맷되지만 직접 입력에는 제한 없음
- **위치:** frontend/src/components/trading/OrderForm.tsx:304
- **영향:** 극단적 소수점 값으로 인한 백엔드 Decimal 연산 비용 증가
- **권장:** 입력 필드에서 소수점 8자리 제한 또는 onBlur 시 .toFixed(8) 적용
- **심각도:** Medium

[VAL-L-01] auth-proxy 컨트롤러 — 인증 관련 body 타입 unknown으로 전달 (Low, 미수정)

- **현상:** auth-proxy.controller.ts 의 register , login , sendSmsCode 등 다수 엔드포인트에서 @Body() body: unknown 으로 body를 수신하여 user-auth 서비스로 그대로 전달. API Gateway 레벨에서 DTO 검증 없이 다운스트림 서비스에 의존
- **위치:** backend/services/api-gateway/src/proxy/auth-proxy.controller.ts:69,97,116,221,237,253,271,287
- **영향:** 잘못된 요청이 다운스트림 서비스까지 전달되어 불필요한 부하 발생
- **권장:** 최소한 register , login 엔드포인트에 Gateway 레벨 DTO 적용 검토
- **심각도:** Low

C. 에러 처리/정보 노출 — 3건

[ERR-M-01] 주문 프록시 — 거래 알림 실패 시 내부 에러 메시지 로깅 (Medium, 미수정)

- 현상: order-proxy.controller.ts:62 에서 sendTradeNotification 실패 시 e.message 를 Logger.warn으로 출력. 프로덕션 환경에서 내부 서비스 URL, 인증 토큰 관련 에러가 로그에 노출될 가능성
- 위치: backend/services/api-gateway/src/proxy/order-proxy.controller.ts:62
- 영향: 로그 수집 시스템에서 민감 정보 노출 가능
- 권장: 에러 메시지를 일반화하거나 e.message 대신 에러 코드만 기록
- 심각도: Medium

[ERR-M-02] any 타입 사용에 의한 타입 안전성 결여 (Medium, 미수정)

- 현상: order-proxy.controller.ts:164-176 에서 as Record<string, any>, (t: any), (u: any) 등 any 타입 사용. portfolio-proxy.controller.ts 에서 (req as Record<string, any>).user?.id 패턴이 14개소 반복(42, 58, 74, 90, 105, 119, 134, 155, 250, 273, 309, 366, 385, 404 행). ai-proxy.controller.ts:48 에서도 동일 패턴 사용
- 위치: backend/services/api-gateway/src/proxy/order-proxy.controller.ts:164-176 , backend/services/api-gateway/src/proxy/portfolio-proxy.controller.ts (14개소), backend/services/api-gateway/src/proxy/ai-proxy.controller.ts:48
- 영향: 예상치 못한 데이터 구조 변경 시 런타임 크래시, 타입 가드 부재로 잘못된 속성 접근 가능
- 권장: @CurrentUser('id') userId: string 데코레이터를 모든 프록시 컨트롤러에 일관 적용 (order-proxy에서는 이미 사용 중)
- 심각도: Medium

[ERR-L-01] 프론트엔드 catch 블록 — 에러 무시 패턴 반복 (Low, 미수정)

- 현상: portfolio/page.tsx:99-101, 113-115 에서 catch { // Error handled by query client } 패턴으로 에러를 무시. 에러 객체를 참조하지 않아 디버깅 시 원인 추적 어려움
- 위치: frontend/src/app/(main)/portfolio/page.tsx:99, 113
- 영향: 네트워크/서버 에러 원인 파악 지연
- 권장: 최소한 catch (e) { console.debug(e); } 또는 에러 추적 서비스 연동
- 심각도: Low

D. XSS/CSP 방어 — 1건

[CSP-M-01] CSP script-src — unsafe-inline + strict-dynamic 공존 (Medium, 미수정)

- 현상: next.config.ts:9 에서 프로덕션 CSP script-src 가 'self' 'unsafe-inline' 'strict-dynamic' 으로 설정. CSP 3.0 지원 브라우저에서 'unsafe-inline' 은 무시되지만, CSP 2.0만 지원하는 브라우저에서 인라인 스크립트 실행 허용. httpOnly 쿠키로 직접적 토큰 탈취는 방지되나, 응답 바디에 토큰이 여전히 노출 (SEC-H-03)되어 XSS 위험이 가중됨
- 위치: frontend/next.config.ts:9
- 영향: CSP 2.0 브라우저에서 XSS 방어력 약화, SEC-H-03과 결합 시 토큰 탈취 가능
- 권장: nonce 기반 CSP로 전환하여 'unsafe-inline' 완전 제거
- 심각도: Medium

E. 인가/권한 검증 — 4건

[AUTH-M-01] 호가창 엔드포인트 — 인증 불필요 노출 (Medium, 미수정)

- **현상:** GET /api/orders/book/:symbol 에 인증 가드가 없음. 호가창 데이터에 사용자들의 주문 가격/수량이 포함
- **위치:** backend/services/api-gateway/src/proxy/order-proxy.controller.ts:208-218
- **영향:** 비인증 스크래퍼가 호가 데이터를 자동 수집하여 시장 미시구조 분석/조작에 활용
- **권장:** 공개 API 여부 명시적 결정 후 비공개 시 JwtAuthGuard 추가, 공개 결정 시 엔드포인트별 Rate Limit 적용
- **심각도:** Medium

[AUTH-L-01] 공개 포트폴리오 API — 보유 자산 절대 수량 노출 (Low, 미수정)

- **현상:** /api/portfolio/public/:userId 를 통해 다른 사용자의 보유 심볼, 수량, 평균가, 수익률을 정확히 조회 가능
- **위치:** backend/services/api-gateway/src/proxy/portfolio-proxy.controller.ts:222-235
- **영향:** 특정 사용자의 정확한 포지션 크기가 노출되어 의도적 반대매매 가능
- **권장:** 보유 비율(%)만 공개하고 절대 수량은 마스킹
- **심각도:** Low

[AUTH-M-02] Admin 페이지 — 미들웨어 레벨 인증 검증 부재 (Medium, 보충 감사 신규)

- **현상:** middleware.ts 에서 /admin 경로에 X-Robots-Tag 와 Cache-Control 보안 헤더만 추가하고, 인증/권한 검증은 수행하지 않음. 관리자 페이지 접근 제어는 프론트엔드 AdminGuard 컴포넌트(client-side)에서만 처리. 비인증 사용자가 /admin/* URL에 직접 접근하면 AdminGuard가 로드되기 전까지 관리자 페이지의 HTML 마크업(스켈레톤 UI 포함)이 일시적으로 노출됨. 백엔드 API는 JwtAuthGuard + AdminRolesGuard 로 보호되어 데이터 유출은 없음
- **위치:** frontend/src/middleware.ts:4-14 , frontend/src/components/layout/AdminGuard.tsx:37-56
- **영향:** Admin 페이지 HTML 구조(사이드바 메뉴, 카드 레이아웃 등)가 SSR 또는 초기 렌더에서 비인증 사용자에게 일시 노출 가능. API 데이터는 보호됨
- **권장:** middleware.ts 에서 access_token 쿠키 존재 여부를 확인하여 미인증 시 /dashboard 로 리다이렉트. 또는 JWT 디코딩(서명 검증 제외)으로 role 확인
- **심각도:** Medium

[MISC-L-01] 공지사항 생성/수정 — Gateway 레벨 AdminRolesGuard 미적용 (Low, 미수정)

- **현상:** AnnouncementProxyController 의 컨트롤러 레벨에 @UseGuards(JwtAuthGuard) 만 적용. POST /api/announcements (생성), PUT /api/announcements/:id (수정), DELETE /api/announcements/:id (삭제), POST /api/announcements/:id/pin (고정) 등 관리자 전용 엔드포인트에 AdminRolesGuard 가 없음. 감사 로그는 추가되었으나 Gateway 레벨 권한 차단은 미적용
- **위치:** backend/services/api-gateway/src/proxy/announcement-proxy.controller.ts:30-31, 125, 156, 193, 217
- **영향:** 관리자가 아닌 인증 사용자가 공지사항 생성/수정/삭제 API를 호출하면 user-auth까지 요청이 전달된 후 거부됨 (불필요한 서비스 부하)
- **권장:** 생성/수정/삭제/고정 토큰 엔드포인트에 AdminRolesGuard 추가
- **심각도:** Low

F. 서비스 간 통신 — 1건

[SVC-M-01] 서비스 간 HTTP 통신 — HTTPS 미적용 (Medium, 미수정)

- **현상:** ProxyService.initClients() 에서 모든 내부 서비스 URL이 http://\${host}:\${port} 형태로 생성. Docker 네트워크 또는 K8s 클러스터 내부에서도 서비스 간 통신이 평문 HTTP로 수행됨
- **위치:** backend/services/api-gateway/src/proxy/proxy.service.ts:70-77
- **영향:** 같은 네트워크에 있는 공격자가 서비스 간 트래픽을 가로채 내부 시크릿 및 사용자 데이터 탈취 가능

- 권장: 프로덕션에서 서비스 메시(Istio/Linkerd) mTLS 적용 또는 서비스 간 HTTPS 사용 검토
 - 심각도: Medium
-

G. 감사 로깅 — 1건

[AUD-M-01] 비밀번호 재설정 — 성공/실패 이벤트 로깅 불충분 (Medium, 미수정)

- 현상: AuthService.resetPassword() 에서 비밀번호 재설정 성공 시 logger.log 만 기록. 실패(잘못된 sessionId, 미인증 세션 등)의 경우 UnauthorizedException 만 발생하고 별도 실패 로그가 없음
 - 위치: backend/services/user-auth/src/application/services/auth.service.ts:668-705
 - 영향: 비밀번호 재설정 악용 시도 패턴 감지 불가
 - 권장: 비밀번호 재설정 성공/실패 시 구조화된 감사 이벤트 기록 (userId, sessionId, IP, timestamp, result)
 - 심각도: Medium
-

H. 기타 — 2건

[FE-M-01] RichEditor 이미지 업로드 — 서버 실패 시 Base64 폴백 유지 (Medium, 미수정)

- 현상: RichEditor.tsx:144-147 에서 서버 업로드 실패 시 editor.chain().focus().setImage({ src: dataUrl }).run() 으로 Base64 data URL 폴백. 대용량 Base64 데이터가 게시글 HTML에 포함되어 DB에 저장됨
- 위치: frontend/src/components/ui/RichEditor.tsx:144-147
- 영향: 서버 불안정 시 DB에 대용량 Base64 저장, 게시글 조회 성능 저하
- 권장: 서버 업로드 실패 시 사용자 에러 알림 후 이미지 삽입 차단
- 심각도: Medium

[FE-M-02] 주문 폼 — displayCurrentPrice 기반 최대 수량 계산 편차 (Medium, 미수정)

- 현상: OrderForm.tsx:321 에서 매수 100% 비율 클릭 시 maxQty = portfolio.cashBalance / displayCurrentPrice 로 계산. displayCurrentPrice 는 환율 변환이 적용된 표시 가격이므로 반올림 오차로 잔액 부족 에러 발생 가능
 - 위치: frontend/src/components/trading/OrderForm.tsx:320-323
 - 영향: 100% 매수 시 환율 변환 오차가 1% 이상일 경우 주문 실패
 - 권장: 최대 수량 계산을 백엔드 USD 가격 기준으로 수행
 - 심각도: Medium
-

I. WebSocket 보안 — 1건

[WS-M-01] WebSocket 인증 토큰 메모리 노출 — XSS 시 탈취 가능 (Medium, 23차 신규)

- 현상: useChatSocket.ts:398 에서 auth: { token } 형태로 액세스 토큰을 Socket.IO handshake에 전달. 이 토큰은 useAuthStore.getState().accessToken 에서 가져오며, Zustand 스토어의 in-memory 값. useWebSocket.ts:117 에서도 auth: accessToken ? { token: accessToken } : undefined 로 동일 패턴 사용. chat.gateway.ts:98-100 에서 client.handshake.auth?.token 으로 추출하여 검증. httpOnly 쿠키로 전환되었으나, WebSocket 인증을 위해 토큰이 여전히 JavaScript 접근 가능한 메모리에 존재하며, XSS 공격 시 useAuthStore.getState().accessToken 으로 직접 읽을 수 있음
- 위치: frontend/src/hooks/useChatSocket.ts:398 , frontend/src/hooks/useWebSocket.ts:117 , backend/services/api-gateway/src/gateway/chat.gateway.ts:96-108
- 영향: XSS 시 메모리 내 토큰 탈취 후 WebSocket 연결 스푸핑 또는 REST API 호출에 악용 가능. 단, 액세스 토큰 만료 (15분)와 쿠키 기반 refresh로 피해 범위는 제한됨

- **권장:** Socket.IO의 `withCredentials: true` + 서버측에서 handshake 쿠키로 인증하도록 전환.
`chat.gateway.ts` 의 `handleConnection` 에서 `client.handshake.headers.cookie` 를 파싱하여 `access_token` 쿠키를 직접 검증
 - **심각도:** Medium
-

J. Git/이력 보안 — 2건 (보충 감사 신규)

[GIT-M-01] .env 파일 Git 이력 잔존 — 개발 환경 자격 증명 노출 (보충 감사 신규)

- **현상:** `.env.development`, `.env.staging`, `.env.production` 파일이 커밋 `05a51fa` 에서 추가되고 커밋 `c80f223` 에서 `.gitignore` 에 추가 후 git 추적에서 제거됨. 현재 작업 트리에서는 `.env.example` 만 추적 중이나, git 이력에 `.env.development` 의 로컬 PostgreSQL 자격 증명(`postgres:postgres`), Redis URL, Kafka 브로커 주소 등이 남아 있음. `.env.production` 은 `CHANGE_ME` 플레이스홀더를 사용하여 실제 시크릿은 미포함. `.env.staging` 도 동일 패턴
- **위치:** Git 이력 — 커밋 `05a51fa3` (추가), `c80f223` (제거)
- **영향:** 리포지토리 접근 권한이 있는 사용자가 `git show 05a51fa:.env.development` 로 개발 환경 자격 증명을 조회 가능. 프로덕션 시크릿은 미노출이나, 내부 서비스 포트 번호, 호스트명 구조, JWT 시크릿 패턴 등 인프라 정보 유추 가능
- **권장:** `git filter-branch` 또는 `BFG Repo-Cleaner` 로 이력에서 `.env.*` 파일 완전 제거. 개발 환경 자격 증명에 실제 서비스와 다르다면 위험도 낮으나, 조직 보안 정책에 따라 이력 정리 필요
- **심각도:** Medium

[GIT-L-01] .env.example 파일 내 내부 서비스 아키텍처 정보 노출 (보충 감사 신규)

- **현상:** `.env.example` 이 git에 추적 중이며, 7개 마이크로서비스의 포트 번호 구조(`USER_AUTH_PORT`, `MARKET_DATA_PORT`, `ORDER_ENGINE_PORT` 등), 서비스 호스트 패턴(`SERVICE_HOST`), 내부 시크릿 변수명(`INTERNAL_SERVICE_SECRET`), JWT 시크릿 변수명 등이 노출됨. 이는 공개 리포지토리에서 공격자에게 서비스 구조 청사진을 제공
 - **위치:** `.env.example` (git tracked)
 - **영향:** 비공개 리포지토리에서는 위험도 낮으나, 공개 전환 시 마이크로서비스 아키텍처와 내부 통신 구조가 노출
 - **권장:** `.env.example` 에서 내부 포트/호스트 관련 변수를 주석으로 설명만 포함하거나, 실제 변수명을 일반화
 - **심각도:** Low
-

K. 파일 업로드/프록시 보안 — 1건 (보충 감사 신규)

[PROXY-L-01] Gateway 레벨 파일명 검증 부재 — 다운스트림 의존 (보충 감사 신규)

- **현상:** `community-proxy.controller.ts:392` 와 `announcement-proxy.controller.ts:71` 의 `serveFile` 엔드포인트에서 `@Param('fileName')` `fileName` 을 검증 없이 `url: /community/uploads/${fileName}` 형태로 다운스트림 서비스에 전달. 다운스트림 `community.controller.ts:615` 에서 `path.basename(fileName)` + null byte 검증으로 path traversal을 방지하고 있으나, Gateway 레벨에서는 별도 검증 없음. 방어 심층(defense-in-depth) 관점에서 Gateway에서도 1차 검증이 필요
- **위치:** `backend/services/api-gateway/src/proxy/community-proxy.controller.ts:392-397`, `backend/services/api-gateway/src/proxy/announcement-proxy.controller.ts:71-76`
- **영향:** 현재는 다운스트림 서비스에서 path traversal을 정상 차단하므로 실질적 위험 없음. 그러나 다운스트림 서비스의 검증 로직이 변경되거나 제거될 경우 path traversal 취약점이 노출
- **권장:** Gateway 프록시에서도 `if (fileName.includes('.') || fileName.includes('\0') || fileName.includes('/')) return res.status(400)` 1차 검증 추가

- 심각도: Low

보충 감사 — 검증 완료 항목 (이슈 아님)

비밀번호 해싱 알고리즘 (정상):

- `auth.service.ts:18` 에서 `bcrypt` 사용, `SALT_ROUNDS` = 환경변수 기반 (기본값 12). `bcrypt` 12 rounds는 현재 기준 충분한 강도 ($2^{12} = 4,096$ iterations). `argon2`로의 전환은 권장 사항이지만 `bcrypt` 12가 취약하지는 않음

계정 잠금 메커니즘 (정상):

- `auth.service.ts:52` 에서 `LOGIN_MAX_ATTEMPTS` = 5 (환경변수로 조정 가능)
- `auth.service.ts:333-350` 에서 SMS 인증 시도 초과 시 `lockUser()` 호출하여 DB에 `lockedAt` 기록
- `auth.service.ts:238-241` 에서 로그인 시 `lockedAt` 확인하여 잠긴 계정 차단

에러 스택 트레이스 노출 (정상):

- `all-exceptions.filter.ts:52-55` 에서 미처리 예외 시 클라이언트에 'Internal server error' 일 반 메시지만 반환, 스택 트레이스는 서버 로그에만 기록
- `proxy.service.ts:165-173` 에서도 연결 실패 시 'Upstream service is temporarily unavailable' 일반 메시지만 반환. 내부 서비스명 미노출 (M-11 패턴 적용 확인)

CSRF 보호 (정상):

- `auth.controller.ts:51,67` 에서 쿠키에 `sameSite: 'lax'` 적용. `auth-proxy.controller.ts:52-54` 에서 Gateway 레벨에서도 SameSite 플래그 추가 보장
- SameSite=Lax는 cross-origin POST 요청에서 쿠키를 전송하지 않으므로 CSRF 공격 차단. 모든 상태 변경 엔드포인트가 POST/PUT/DELETE를 사용하므로 잔여 CSRF 벡터 없음

오픈 리다이렉트 (정상):

- 로그인 성공 시 `router.push('/dashboard')` 하드코딩. 로그아웃 시 `window.location.replace('/login')` 하드코딩
- 사용자 입력 기반 리다이렉트 파라미터(`returnUrl`, `next` 등) 미사용. 오픈 리다이렉트 벡터 없음

미들웨어 Admin 라우트 보호 (설계 의도 확인):

- `middleware.ts` 에서 `/admin` 경로에 `X-Robots-Tag` 와 `Cache-Control` 헤더만 추가, 인증 검증 없음
- 관리자 인증은 프론트엔드 `AdminGuard` 컴포넌트(client-side)에서 처리하고, 백엔드 API는 `JwtAuthGuard` + `AdminRolesGuard` 로 보호
- Next.js middleware에서 쿠키 기반 인증 검증을 추가하면 SSR 시점에서도 비인증 접근을 차단할 수 있으나, 현재 API 레벨 보호가 작동하므로 데이터 유출 위험은 없음 (UX 개선 수준)

종합 평가

23차 시스템 감사(보충 감사 포함)에서 총 19건의 이슈를 발견하였습니다. 22차 대비 1건이 수정되었고(SEC-H-02), 13건이 미수정 상태이며, 23차 본 감사에서 3건, 보충 감사에서 3건의 신규 이슈가 발견되었습니다.

수정 현황 (1건 수정)

- **SEC-H-02** (High → 수정): API Gateway 로그아웃 시 `access_token` 쿠키 삭제 — `auth-proxy.controller.ts:189-190` 에서 `res.clearCookie('access_token', { path: '/' })` 추가 확인. user-auth의 `auth.controller.ts:214-216` 에서도 양쪽 쿠키 삭제 확인

미수정 13건

- VAL-M-02, ERR-M-01, ERR-M-02, ERR-L-01, CSP-M-01, AUTH-M-01, AUTH-L-01, SVC-M-01, AUD-M-01, FE-M-01, FE-M-02, MISC-L-01, VAL-L-01

23차 신규 발견 (본 감사 3건 + 보충 감사 3건)

1. 응답 바디에 **accessToken** 노출 (SEC-H-03, High) — httpOnly 쿠키 마이그레이션 회귀. WebSocket 호환을 위해 응답 바디에 토큰이 여전히 포함되어 XSS 시 탈취 가능
2. **refresh** 엔드포인트 **Rate Limit** 미적용 (SEC-M-01, Medium) — 모든 인증 엔드포인트 중 유일하게 Rate Limit 없음
3. **WebSocket** 인증 토큰 메모리 노출 (WS-M-01, Medium) — WebSocket 인증을 위해 액세스 토큰이 JavaScript 접근 가능한 Zustand 메모리에 존재
4. **.env** 파일 **Git** 이력 잔존 (GIT-M-01, Medium, 보충 감사) — 개발 환경 자격 증명이 git 이력에 잔존.
 .gitignore 추가로 현재 추적은 중단되었으나 이력 미정리
5. **.env.example** 내부 **아키텍처** 정보 (GIT-L-01, Low, 보충 감사) — 7개 마이크로서비스 포트/호스트 구조가 .env.example 에 노출
6. **Gateway** 레벨 **파일명** 검증 부재 (PROXY-L-01, Low, 보충 감사) — 파일 다운로드 프록시에서 다운스트림 검증에 의존, Gateway 1차 검증 미적용

httpOnly 쿠키 마이그레이션 회귀 분석

22차에서 수정된 SEC-C-01(JWT sessionStorage 저장)은 정상적으로 적용되었습니다:

- auth.ts 에서 partialize 로 accessToken이 sessionStorage에서 제외됨 (확인)
- jwt.strategy.ts 에서 쿠키 우선 추출 정상 동작 (확인)
- api.ts 에서 withCredentials: true 설정되어 쿠키 자동 전송 (확인)
- jwt-auth.guard.ts:37-39 에서 쿠키→Authorization 헤더 복원 패턴은 보안상 안전함 (서버 내부에서만 동작하며 외부 클라이언트는 접근 불가)

그러나 WebSocket 호환을 위해 도입된 우회 경로(응답 바디 토큰 + 메모리 저장)가 httpOnly 쿠키의 보안 이점을 부분적으로 무효화하고 있어 SEC-H-03으로 보고합니다.

JwtAuthGuard 쿠키→헤더 복원 보안 분석

jwt-auth.guard.ts:36-39 에서 request.headers.authorization = Bearer \${request.cookies.access_token} 패턴은:

- 서버 내부 request 객체에만 적용되며 클라이언트에 노출되지 않음 (안전)
- 프록시 컨트롤러가 다운스트림 서비스에 Authorization 헤더를 전달하는 데 필요한 정상 동작 (안전)
- CORS allowedHeaders 에서 내부 헤더가 이미 제외되어 외부 조작 불가 (안전)

CORS 설정 검증

main.ts:103-111 에서:

- origin : 환경 변수 기반 화이트리스트 (정상)
- credentials: true : httpOnly 쿠키 전송에 필수 (정상)
- allowedHeaders : ['Content-Type', 'Authorization', 'Accept'] 만 허용, 내부 헤더 제외 (정상)

AI 엔드포인트 Rate Limiting 효과성 검증

ai-proxy.controller.ts:21 에서 컨트롤러 레벨 @Throttle({ default: { ttl: 60000, limit: 10 } }) 적용 확인. 모든 AI 엔드포인트(/signals , /portfolio-analysis , /news-summary)에 동일하게 분당 10회

제한 적용. 효과적으로 작동하나, 사용자별이 아닌 IP별 제한이므로 공유 네트워크 환경에서 다른 사용자에게 영향을 줄 수 있음 (개선 권장 수준).

우선 수정 권장 순서

1. SEC-H-03 (High) — 응답 바디 토큰 제거 + WebSocket 쿠키 인증 전환
2. SEC-M-01 (Medium) — refresh 엔드포인트 Rate Limit 추가 (1줄 추가로 해결)
3. WS-M-01 (Medium) — WebSocket 쿠키 기반 인증 전환 (SEC-H-03과 동시 해결)
4. GIT-M-01 (Medium) — git 이력에서 .env 파일 완전 제거 (BFG Repo-Cleaner)
5. CSP-M-01 (Medium) — nonce 기반 CSP 전환
6. ERR-M-02 (Medium) — `@CurrentUser` 데코레이터 일관 적용으로 any 타입 제거

본 보고서는 자동 생성된 감사 결과이며, 수정 사항은 포함되지 않습니다.